JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.ref

Class SoftReference<T>

java.lang.Object
    java.lang.ref.Reference<T>
        java.lang.ref.SoftReference<T>

```
public class SoftReference<T>
extends Reference<T>
```

Soft reference objects, which are cleared at the discretion of the garbage collector in response to memory demand. Soft references are most often used to implement memory-sensitive caches.

Suppose that the garbage collector determines at a certain point in time that an object is softly reachable. At that time it may choose to clear atomically all soft references to that object and all soft references to any other softly-reachable objects from which that object is reachable through a chain of strong references. At the same time or at some later time it will enqueue those newly-cleared soft references that are registered with reference queues.

All soft references to softly-reachable objects are guaranteed to have been cleared before the virtual machine throws an `OutOfMemoryError`. Otherwise no constraints are placed upon the time at which a soft reference will be cleared or the order in which a set of such references to different objects will be cleared. Virtual machine implementations are, however, encouraged to bias against clearing recently-created or recently-used soft references.

Direct instances of this class may be used to implement simple caches; this class or derived subclasses may also be used in larger data structures to implement more sophisticated caches. As long as the referent of a soft reference is strongly reachable, that is, is actually in use, the soft reference will not be cleared. Thus a sophisticated cache can, for example, prevent its most recently used entries from being discarded by keeping strong referents to those entries, leaving the remaining entries to be discarded at the discretion of the garbage collector.

Since:

1.2

Constructor Summary

Constructors

Constructor	Description
`SoftReference(T referent)`	Creates a new soft reference that refers to the given object.
`SoftReference(T referent, ReferenceQueue<? super T> q)`	Creates a new soft reference that refers to the given object and is registered with the given queue.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`T`	`get()`	Returns this reference object's referent.

Methods inherited from class java.lang.ref.Reference

`clear`, `clone`, `enqueue`, `isEnqueued`

Methods inherited from class java.lang.Object

equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

SoftReference

```
public SoftReference(T referent)
```

Creates a new soft reference that refers to the given object. The new reference is not registered with any queue.

Parameters:

`referent` - object the new soft reference will refer to

SoftReference

```
public SoftReference(T referent,
                     ReferenceQueue<? super T> q)
```

Creates a new soft reference that refers to the given object and is registered with the given queue.

Parameters:

`referent` - object the new soft reference will refer to

`q` - the queue with which the reference is to be registered, or `null` if registration is not required

Method Detail

get

```
public T get()
```

Returns this reference object's referent. If this reference object has been cleared, either by the program or by the garbage collector, then this method returns `null`.

Overrides:

`get` in class `Reference<T>`

Returns:

The object to which this reference refers, or null if this reference object has been cleared

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.